

**Автономная некоммерческая организация профессиональная
образовательная организация
«Колледж экономики, права и информационных технологий»
(АНО ПОО «КЭПиИТ»)**

КУРСОВОЙ ПРОЕКТ

**по теме: «Разработка веб-приложения потокового воспроизведения
аудиоконтента «SIGNAL»»**

Специальность: 09.02.07 Информационные технологии и программирование

Квалификация: Разработчик веб и мультимедийных приложений

Обучающийся: _____ / Шевченко Н.Е.
(подпись) (Фамилия И.О.)

Руководитель курсового проекта: _____ / Двораковский Б.М.
(подпись)

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. ПРЕДПРОЕКТНОЕ ИССЛЕДОВАНИЕ	5
1.1 Описание предметной области	5
1.2 Описание технологии обработки задач	8
1.3 Информационно-логическая модель системы	11
1.4 Характеристика инструментальных средств разработки	18
ГЛАВА 2. РАБОЧИЙ ПРОЕКТ	23
2.1 Разработка веб-приложения	23
2.2 Отладка и тестирование веб-приложения	30
2.3 Публикация веб-приложения	34
2.4 Руководство программиста	36
ЗАКЛЮЧЕНИЕ	41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	43
ПРИЛОЖЕНИЕ	45

ВВЕДЕНИЕ

В современном мире сложно представить повседневную деятельность и досуг человека без использования информационно-компьютерных технологий. Мультимедийные сервисы и потоковое воспроизведение аудиоданных прочно вошли в жизнь общества, став неотъемлемой частью цифровой среды. Одной из ключевых причин бурного развития веб-технологий выступает возможность предоставления пользователю быстрого и удобного доступа к медиаконтенту без необходимости его предварительного скачивания на локальные устройства.

Информационные процессы существенно трансформируют структуру потребления цифрового медиа. В условиях высокой динамики информационного обмена пользователям требуется мгновенный доступ к любимым трекам, плейлистам и радиостанциям с любого устройства — будь то персональный компьютер или смартфон. Создание независимых веб-приложений открывает широкие возможности оптимизации доступа к мультимедиа, исключая зависимость от тяжеловесного программного обеспечения и закрытых коммерческих экосистем.

Однако в настоящее время сфера музыкального стриминга сталкивается с рядом серьезных ограничений. Большинство популярных платформ перегружены навязчивой рекламой, устанавливают жесткие лимиты функционала для бесплатных учетных записей и вводят дорогостоящие платные подписки. Дополнительным барьером выступают региональные блокировки, вынуждающие пользователей прибегать к сторонним VPN-сервисам, что влечет за собой падение скорости соединения, увеличение задержек буферизации и нестабильность аудиопотока.

Кроме того, пользователям приходится выполнять множество лишних действий, распределяя задачи между разрозненными сервисами: отдельно для локальной аудиотеки, отдельно для контента видеохостингов и отдельно для онлайн-радио. Прямое обращение к внешним видеоплатформам сопряжено с частым появлением защитных капч, блокировками запросов и

повышенным расходом интернет-трафика из-за принудительной передачи видеоряда, в то время как пользователю требуется исключительно звуковая дорожка.

В процессе анализа предметной области было выявлено, что проблемным местом является отсутствие единого легковесного веб-ресурса, объединяющего возможности свободного поиска треков, интеграции плейлистов и прослушивания радиостанций без рекламы, капч и необходимости использования VPN.

В связи с этим было принято решение спроектировать и разработать независимый кроссплатформенный веб-плеер «SIGNAL» (Minimalist Audio Player & Stream Engine). Ключевой особенностью сервиса выступает архитектура серверного проксирования: сервер берет на себя взаимодействие с медиаисточниками, извлечение аудиодорожек и передачу оптимизированного потока клиенту, нивелируя региональные ограничения и гарантируя высокую стабильность воспроизведения.

Основной целью курсового проекта является проектирование и разработка клиент-серверного веб-ресурса «SIGNAL» для потокового воспроизведения аудиоконтента с механизмом серверного проксирования, обеспечивающего свободный доступ к медиатеке без рекламы и региональных ограничений.

Для достижения поставленной цели необходимо решить следующие основные задачи:

- 1. Провести исследование предметной области потокового аудиовещания и сравнительный анализ существующих аналогов;
- 2. Сформулировать функциональные и нефункциональные требования к разрабатываемому веб-плееру;
- 3. Спроектировать клиент-серверную архитектуру веб-приложения и спецификацию межсервисного интерфейса REST API;
- 4. Разработать высокопроизводительную серверную часть на языке Rust с использованием фреймворка Axum и асинхронной среды Tokio;

- 5. Реализовать серверный модуль парсинга метаданных и прямого проксирования аудиопотоков на основе утилит yt-dlp и FFmpeg;
- 6. Спроектировать и разработать адаптивный пользовательский интерфейс (SPA) на базе фреймворка Angular с интеграцией плеера и каталога мировых радиостанций;
- 7. Реализовать подсистему синхронизации медиатеки с сервером, локальный экспорт и импорт данных в формате JSON, а также настроить контейнеризацию проекта посредством Docker Compose.

Объектом исследования являются процессы потоковой передачи, обработки и воспроизведения цифрового аудиоконтента в веб-среде.

Предметом исследования выступают методы, алгоритмы и программные средства разработки высоконагруженного клиент-серверного веб-плеера с серверным проксированием потоков.

Методы исследования включают методы системного анализа, функционально-структурного моделирования бизнес-процессов в нотации IDEF0, проектирования реляционных баз данных, методы асинхронного сетевого программирования и компонентно-ориентированной веб-разработки.

Практическая значимость работы заключается в создании готового к практической эксплуатации независимого веб-сервиса «SIGNAL», оптимизирующего доступ к медиаконтенту, устраняющего потребность в VPN и сторонних подписках, а также снижающего нагрузку на аппаратные ресурсы пользовательских устройств.

ГЛАВА 1 ПРЕДПРОЕКТНОЕ ИССЛЕДОВАНИЕ

1.1 Описание предметной области

Предметная область курсового проекта охватывает технологии потокового мультимедиа, архитектуры клиент-серверного взаимодействия и методы распределения сетевой нагрузки при доставке медиаконтента конечному пользователю [1]. Потоковое аудио (аудиостриминг) представляет собой непрерывную передачу аудиоданных от сервера к клиентскому приложению через сеть Интернет, при которой воспроизведение начинается непосредственно во время загрузки первых чанков (порций) данных, не дожидаясь завершения передачи файла целиком.

В таблице 1 представлен сравнительный анализ существующих популярных музыкальных платформ и разрабатываемого сервиса «SIGNAL».

Таблица 1. Сравнительный анализ стриминговых сервисов

Критерий сравнения	Spotify	Яндекс.Музыка	YouTube Music	SIGNAL
Наличие рекламы	Присутствует в бесплатной версии	Присутствует без подписки	Встроена в аудио и видеоряд	Полностью отсутствует
Необходимость платной подписки	Обязательна для комфортного использования	Обязательна для прослушивания	Обязательна для фонового режима	Бесплатный открытый сервис
Доступность без VPN	Заблокирован в РФ	Доступен	Замедлен / заблокирован	Доступен (проксирование сервером)
Нагрузка на интернет-трафик	Средняя	Средняя	Высокая (передача видео)	Минимальная (только аудиопоток)
Каталог онлайн-радио	Ограничен	Только тематические станции	Отсутствует	Мировой каталог SomaFM / Icecast
Синхронизация и экспорт базы	Закрытый формат	Закрытый формат	Закрытый формат	Полный экспорт/импорт в JSON и БД
Требования к ресурсам	Высокие (Electron-клиент)	Высокие (тяжелый веб-код)	Очень высокие	Легковесный SPA на Angular + Rust

Как демонстрирует сравнительный анализ в таблице 1, современные платформы накладывают на пользователя серьезные коммерческие и технические барьеры. Проект «SIGNAL» призван решить данные проблемы за счет реализации независимого сервера-прокси, берущего на себя маршрутизацию запросов и гарантирующего слушателю чистый аудиопоток с минимальными задержками.

Категории пользователей разрабатываемого веб-приложения:

- любители музыки и подкастов всех возрастных категорий (12+), ценящие минимализм и высокую скорость загрузки интерфейса;
- слушатели онлайн-радиостанций со всего мира, интересующиеся эстетикой, электронной музыкой, джазом и инди-направлениями;
- пользователи с нестабильным или лимитированным интернет-соединением, для которых критично отсутствие лишнего видеотрафика;
- пользователи, столкнувшиеся с блокировками зарубежных сервисов стриминга и не желающие постоянно использовать сторонние VPN.

1.2 Описание технологии обработки задач

Технология обработки задач веб-приложения «SIGNAL» основана на клиент-серверной архитектуре с асинхронной обработкой потоковых сетевых запросов. Входная информация делится на два основных потока: пользовательские управляющие данные (запросы поиска, ссылки на медиа, параметры сортировки, учетные данные) и потоковые мультимедиа-данные, поступающие от внешних провайдеров (серверов радиовещания и медиахостингов).

В таблице 2 приведено подробное описание входной информации системы.

Таблица 2. Перечень и спецификация входных данных

Идентификатор	Наименование параметра	Тип данных	Диапазон / Формат	Назначение
INP-01	Поисковый запрос	Строковый (String)	1-255 символов (UTF-8)	Поиск треков по названию или

				исполнителю
INP-02	URL медиаисточника	Строковый (URL)	Валидный http/https URL	Прямое воспроизведение видео/аудио потока
INP-03	Логин пользователя	Строковый (String)	3-32 символов [a-zA-Z0-9_]	Идентификация учетной записи в системе
INP-04	Пароль пользователя	Строковый (String)	Не менее 6 символов	Аутентификация (хэшируется в Argon2id)
INP-05	HTTP Range Header	Строковый (Header)	bytes=start-end	Запрос части аудиопотока для перемотки (Seek)
INP-06	JSON-файл медиаотеки	Файл (JSON)	Структурирован ный JSON	Импорт локальных плейлистов и треков
INP-07	Громкость / Позиция	Числовой (Float)	0.0 - 1.0 / 0 - Duration	Управление аудиотрактом HTML5 Audio

В таблице 3 приведено описание выходной информации, генерируемой сервером и отображаемой клиентским интерфейсом.

Таблица 3. Перечень и спецификация выходных данных

Идентификатор	Наименование сообщения / формы	Тип данных	Формат вывода	Получатель
OUT-01	Проксированный аудиопоток	Бинарный поток (Stream)	audio/mpeg, audio/webm, m3u8	Клиентский HTML5 Audio Element
OUT-02	Каталог результатов поиска	Структура JSON	Массив объектов TrackMeta	Компонент поиска Angular SPA
OUT-03	JWT-токен авторизации	Строковый токен	Bearer token (RFC 7519)	Локальное хранилище браузера клиента
OUT-04	Синхронизирова нная медиатека	Структура JSON	Объект {tracks, playlists}	Сервис управления библиотекой Angular
OUT-05	Спектрограмма / Эквалайзер	Графический буфер	Canvas 2D / Web Audio API	Экранная форма плеера
OUT-06	Статистика SIGNAL Wrapped	Аналитический отчет JSON	Массив топ- жанров и треков	Модальное окно аналитики

В таблице 4 приведен подробный перечень функциональных требований к системе.

Таблица 4. Перечень функциональных требований

№	Функциональное требование	Описание требования
ФТ-01	Поиск аудиокomпозиций	Возможность выполнения полнотекстового поиска треков без необходимости ввода защитных капч.
ФТ-02	Проксирование аудиопотока	Серверное извлечение аудиодорожки и потоковая трансляция с поддержкой частичной загрузки (HTTP Range).
ФТ-03	Воспроизведение онлайн-радио	Поддержка каталога мировых радиостанций с фильтрацией по странам, стилям и битрейту.
ФТ-04	Управление воспроизведением	Функции Play, Pause, Next, Previous, Shuffle (случайный порядок), Repeat (повтор) и плавная смена громкости.
ФТ-05	Управление плейлистами	Создание, редактирование, удаление пользовательских списков воспроизведения и группировка треков.
ФТ-06	Синхронизация с сервером	Автоматическая передача медиатеки в облачную СУБД SQLite для авторизованных пользователей.
ФТ-07	Локальный экспорт/импорт	Резервное копирование медиатеки в файл формата JSON и восстановление данных без авторизации.
ФТ-08	Аналитика прослушиваний	Сбор статистики прослушанных композиций и генерация ежегодного отчета «SIGNAL Wrapped».

В таблице 5 представлены эксплуатационные (нефункциональные) требования к разрабатываемому программному комплексу.

Таблица 5. Эксплуатационные требования к системе

№	Категория требования	Спецификация требования
НТ-01	Производительность	Время начала воспроизведения аудиопотока

		не должно превышать 1.5 сек при скорости сети от 2 Мбит/с.
НТ-02	Потребление памяти сервером	Базовое потребление оперативной памяти серверным процессом на Rust не более 60 МБ.
НТ-03	Безопасность учетных данных	Хэширование пользовательских паролей по стойкому криптографическому алгоритму Argon2id.
НТ-04	Авторизация запросов	Защита конечных точек REST API с использованием цифровых токенов JWT (JSON Web Tokens).
НТ-05	Кроссбраузерность	Корректная работа интерфейса в современных браузерах Google Chrome, Mozilla Firefox, Apple Safari, MS Edge.
НТ-06	Адаптивность верстки	Поддержка экранов смартфонов, планшетов и широкоформатных мониторов (от 360px до 4K).

1.3 Информационно-логическая модель системы

Для формализации информационных процессов и детального проектирования структуры веб-приложения «SIGNAL» было проведено функциональное моделирование с использованием CASE-средства Ramus в нотации IDEF0 [2]. Моделирование в стандарте IDEF0 позволяет описать бизнес-процессы системы в виде иерархических взаимосвязанных функциональных блоков с четким разделением входных и выходных потоков, управляющих воздействий и исполнительных механизмов.

На рисунке 1 представлена контекстная диаграмма верхнего уровня A0 «Организация потокового воспроизведения аудиоконтента».

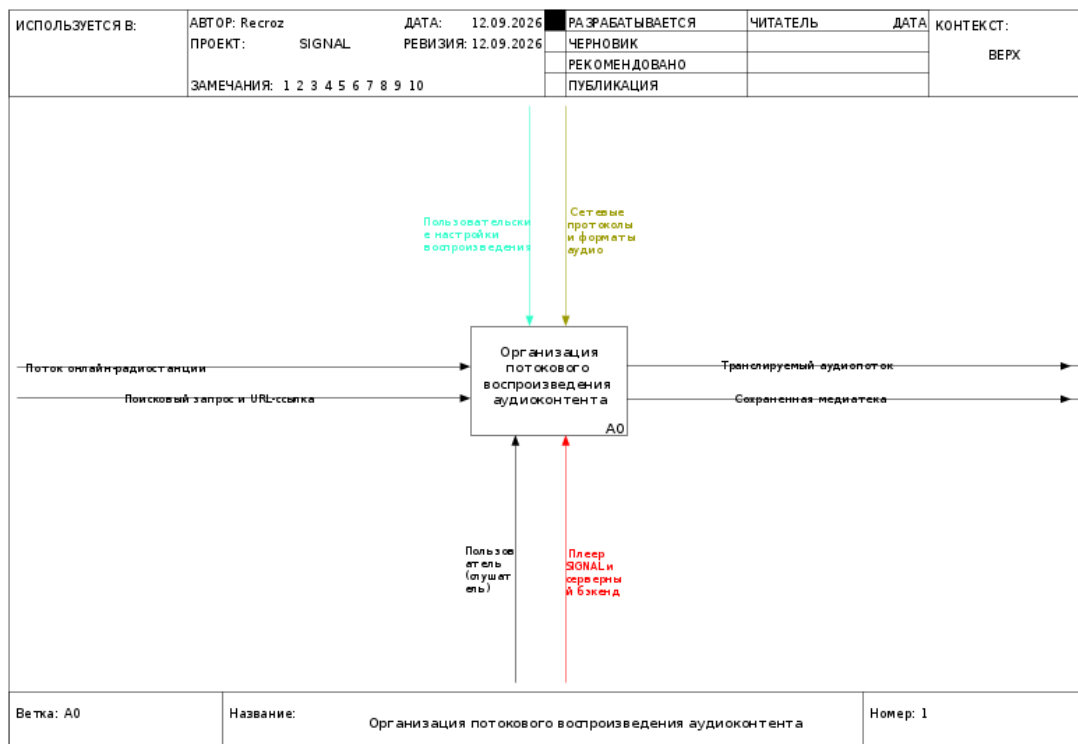


Рисунок 1. Контекстная диаграмма IDEF0 A0 «Организация потокового воспроизведения аудиоконтента»

Контекстная диаграмма A0 отражает систему в целом. В качестве входных данных (стрелки слева) выступают «Поток онлайн-радиостанции» и «Поисковый запрос и URL-ссылка». Управляющими воздействиями (стрелки сверху) являются «Пользовательские настройки воспроизведения» и «Сетевые протоколы и форматы аудио». В роли механизмов исполнения (стрелки снизу) выступают «Пользователь (слушатель)» и «Плеер SIGNAL и серверный бэкэнд». Результатом выполнения процесса (выходные стрелки справа) являются «Транспируемый аудиопоток» и «Сохраненная медиатека».

На рисунке 2 представлена диаграмма декомпозиции первого уровня функционального блока A0.

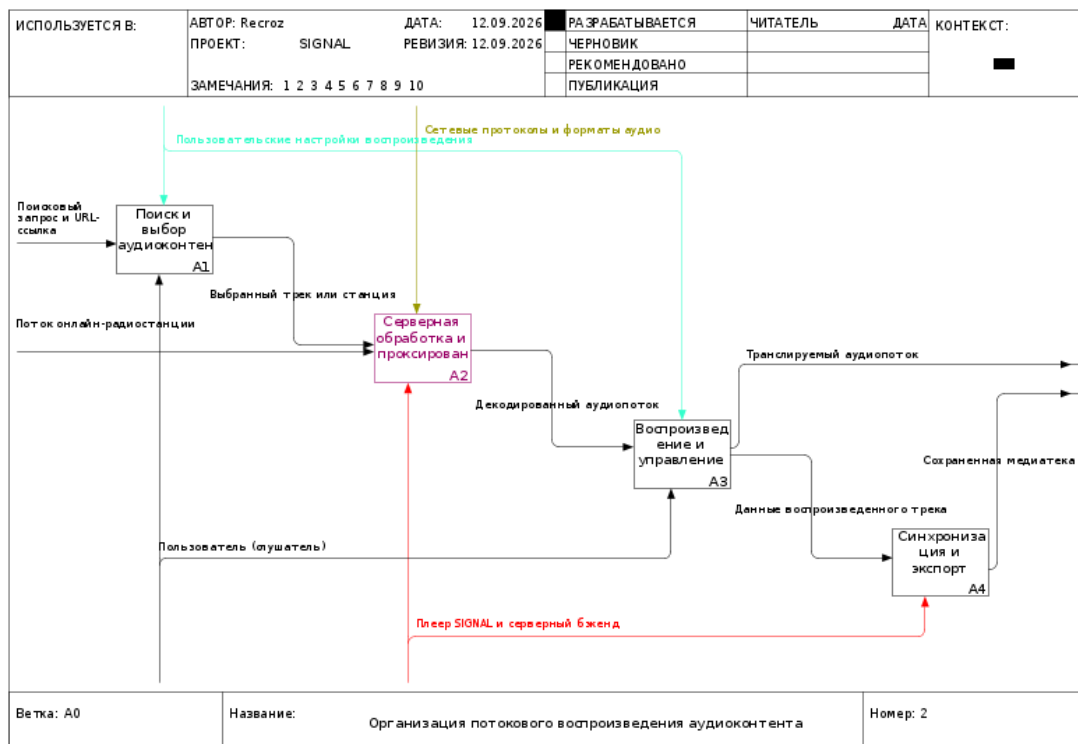


Рисунок 2. Диаграмма декомпозиции IDEF0 A0 первого уровня

В ходе декомпозиции процесса A0 выделены 4 ключевые функциональные подсистемы:

- A1: «Поиск и выбор аудиоконтента» — обработка запросов слушателя и выбор источника;
- A2: «Серверная обработка и проксирование» — обращение к медиаисточникам, извлечение аудиодорожки и буферизация;
- A3: «Воспроизведение и управление» — передача потока в клиентский аудиотракт, визуализация и управление;
- A4: «Синхронизация и экспорт» — фиксация истории прослушиваний, управление плейлистами и резервное копирование.

На рисунке 3 представлена функциональная декомпозиция блока A1 «Поиск и выбор аудиоконтента».

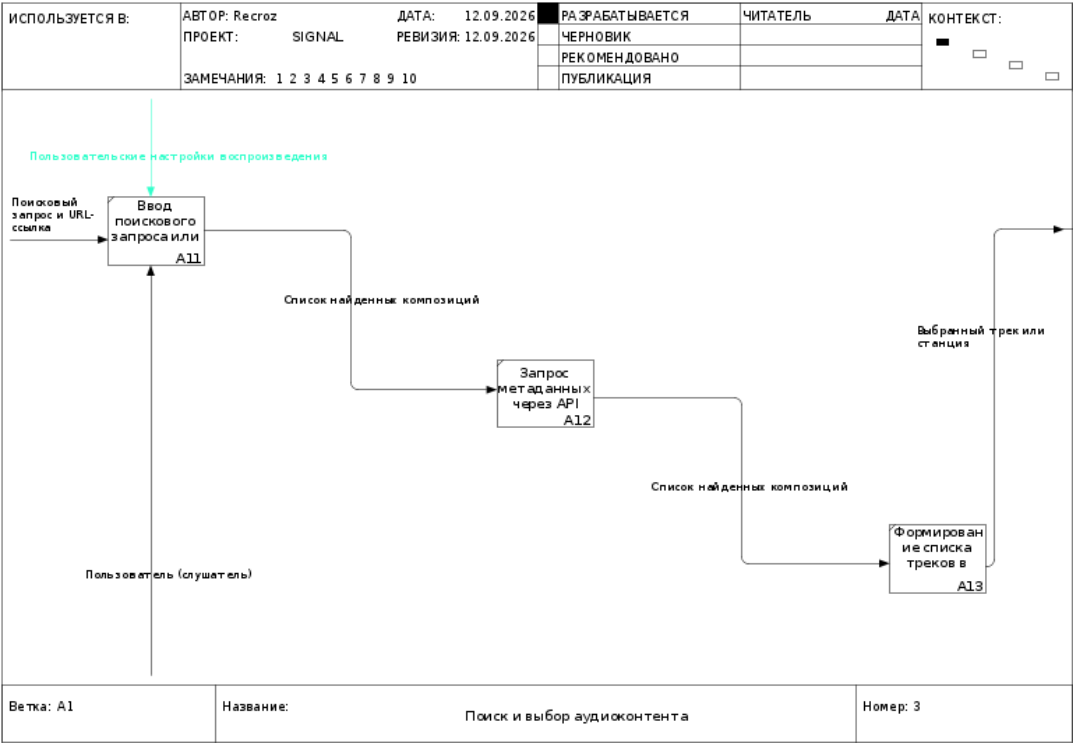


Рисунок 3. Декомпозиция функционального блока A1 «Поиск и выбор аудиоконтента»

Блок A1 декомпозирован на следующие элементарные процессы: A11 «Ввод поискового запроса или URL», в рамках которого пользователь инициирует действие через интерфейс; A12 «Запрос метаданных через API», при котором фронтенд обращается к бэкенду; A13 «Формирование списка треков», в ходе которого формируется интерактивный каталог композиций.

На рисунке 4 приведена диаграмма декомпозиции блока A2 «Серверная обработка и проксирование потока».

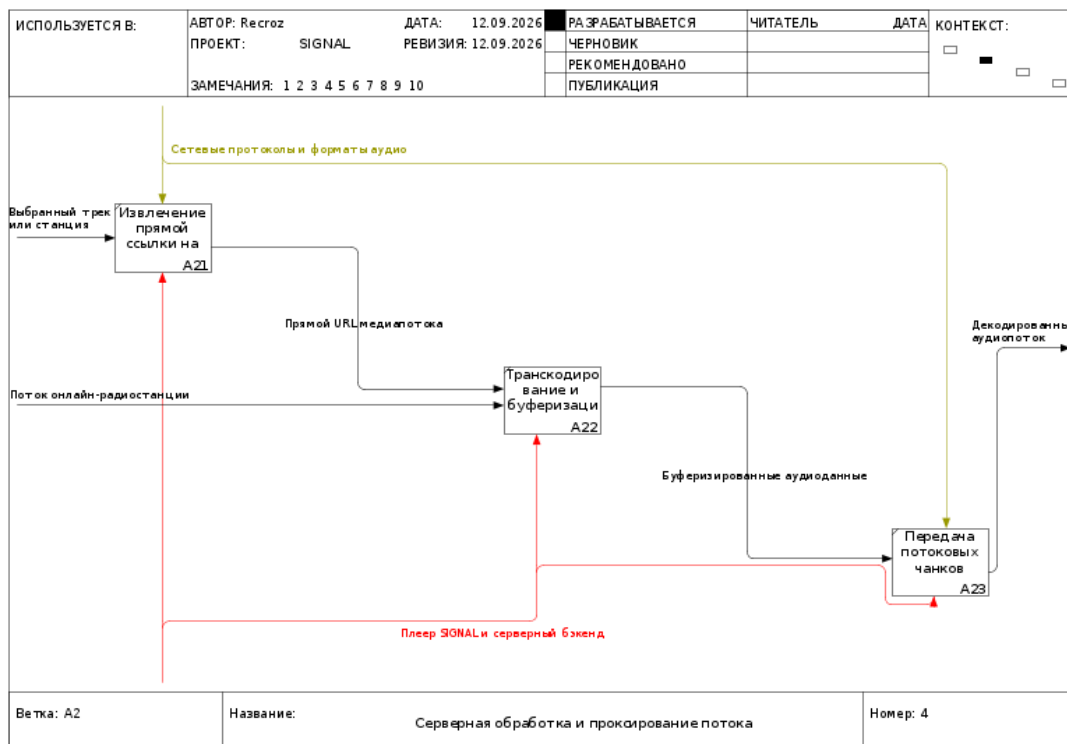


Рисунок 4. Декомпозиция функционального блока A2 «Серверная обработка и проксирование потока»

Блок A2 включает процессы: A21 «Извлечение прямой ссылки на аудиопоток» через утилиту `yt-dlp` без скачивания видеоряда; A22 «Транскодирование и буферизация» аудиоданных с использованием асинхронных буферов `Tokio`; A23 «Передача потоковых чанков» в HTTP-ответ клиенту с поддержкой докачки `Range`.

На рисунке 5 представлена функциональная декомпозиция блока A3 «Воспроизведение и управление аудиотрансляцией».

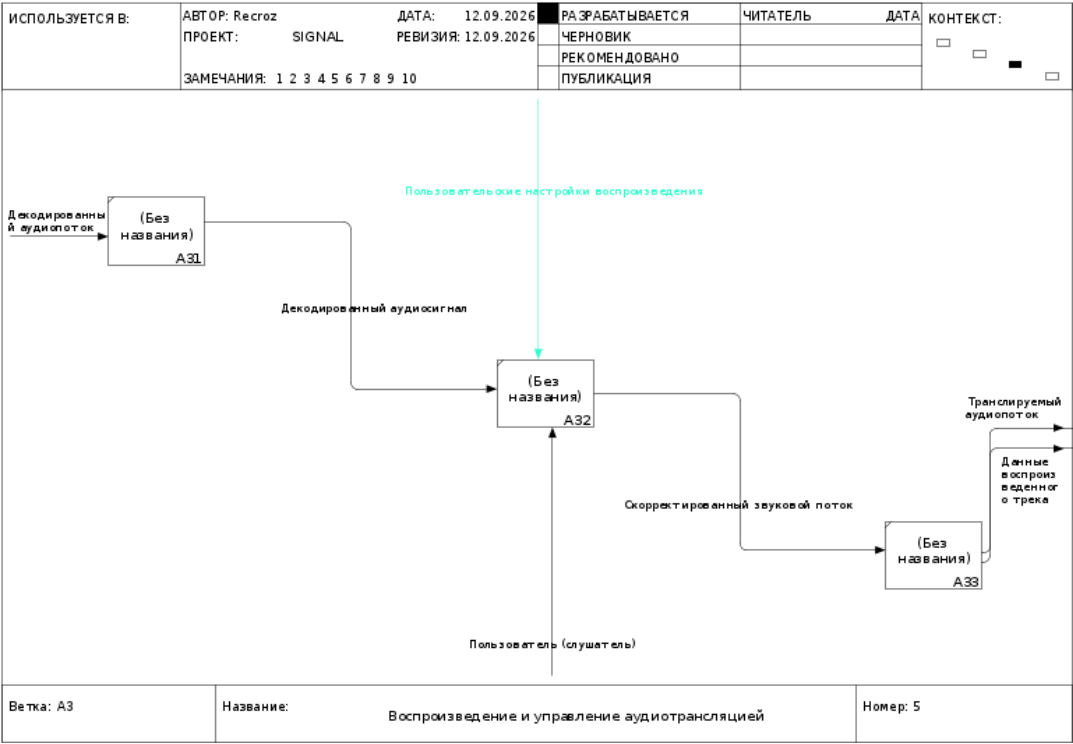


Рисунок 5. Декомпозиция функционального блока АЗ «Воспроизведение и управление аудиотрансляцией»

Блок АЗ детализирует процесс декодирования аудиосигнала браузером, корректировку параметров воспроизведения (громкость, баланс, эквалайзер, фильтры) и непосредственную трансляцию звука на аудиоустройства пользователя с обновлением таймлайна.

На рисунке 6 приведена диаграмма декомпозиции блока А4 «Синхронизация и экспорт медиатеки».

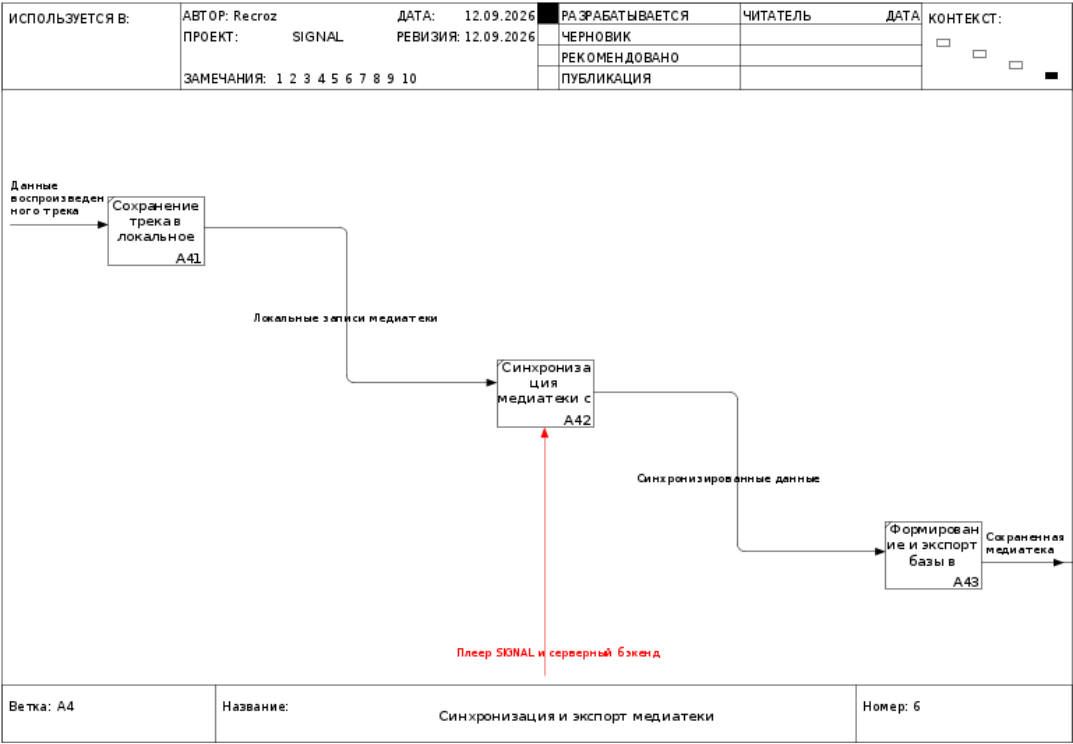


Рисунок 6. Декомпозиция функционального блока A4 «Синхронизация и экспорт медиатеки»

Блок A4 охватывает шаги сохранения трека в локальное хранилище браузера (A41), облачную синхронизацию медиатеки с базой данных сервера по защищенному протоколу (A42) и локальный экспорт всей базы в JSON-файл (A43).

На рисунке 7 представлена диаграмма вариантов использования (Use Case Diagram), иллюстрирующая взаимодействие пользователя и серверной подсистемы.



Рисунок 7. Диаграмма вариантов использования веб-плеера «SIGNAL»

Диаграмма прецедентов наглядно демонстрирует границы ответственности: пользователь взаимодействует с элементами интерфейса (поиск, воспроизведение, создание плейлистов, экспорт данных), а серверная подсистема обеспечивает фоновую валидацию, извлечение потоков и управление базой данных.

На рисунке 8 приведена инфологическая ER-диаграмма структуры базы данных SQLite.

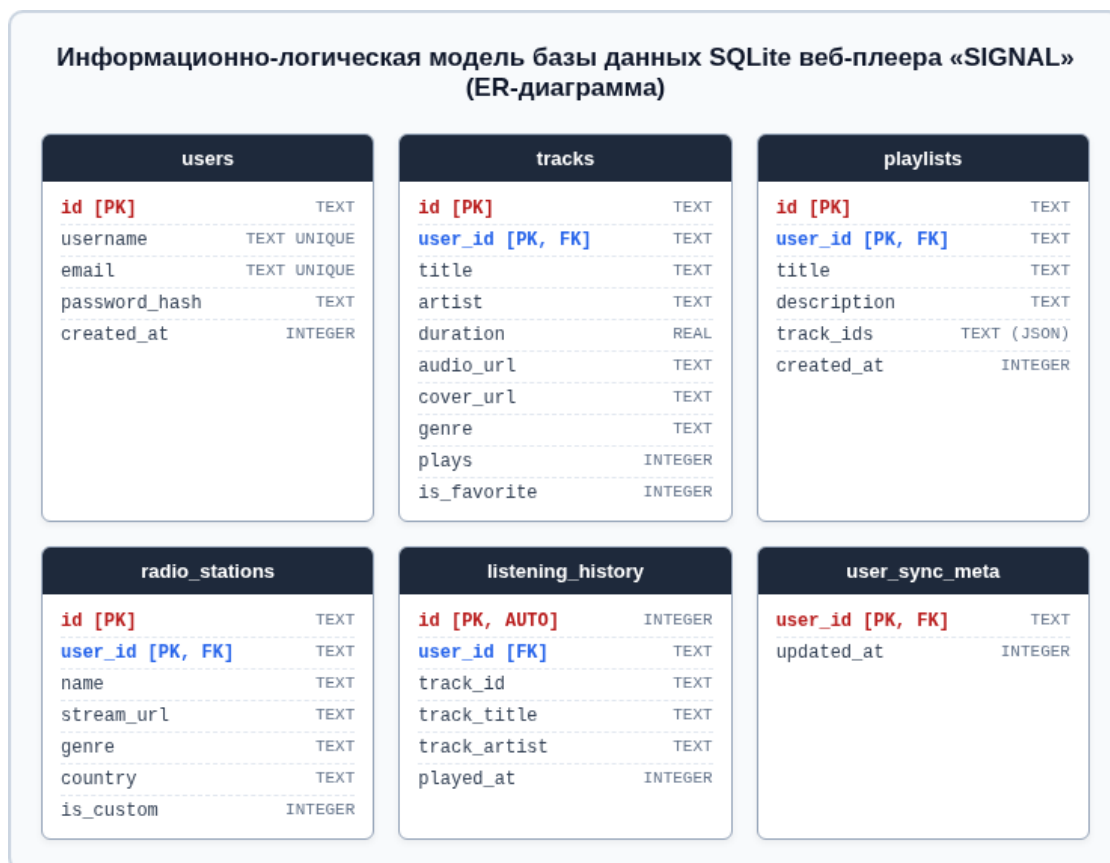


Рисунок 8. Информационно-логическая модель базы данных SQLite (ER-диаграмма)

ER-диаграмма отражает состав и связи таблиц базы данных: «users» (аккаунты), «tracks» (сохраненные аудиокomпозиции), «playlists» (пользовательские плейлисты), «radio_stations» (каталог радиостанций) и «listening_history» (история прослушиваний для Wrapped-аналитики). Связи построены по внешнему ключу user_id с каскадным удалением.

На рисунке 9 изображена общая архитектурная схема веб-приложения «SIGNAL».



Рисунок 9. Архитектурная схема взаимодействия компонентов веб-приложения «SIGNAL»

1.4 Характеристика инструментальных средств разработки

Выбор стека технологий для реализации клиент-серверного мультимедийного веб-ресурса определялся высокими требованиями к производительности при обработке непрерывных потоковых данных, надежности серверной части и отзывчивости пользовательского интерфейса [3].

В таблице 6 представлен сравнительный анализ языков и платформ серверной разработки.

Таблица 6. Сравнительный анализ серверных технологий

Критерий оценки	Rust (Axum + Tokio)	Node.js (Express)	Python (FastAPI)
Производительность и задержки	Максимальная (нативный машинный код)	Средняя (V8, Single-thread EventLoop)	Умеренная (GIL, интерпретатор)
Потребление памяти (RAM)	Минимальное (20-60 МБ)	Среднее (150-300 МБ)	Повышенное (200-400 МБ)
Модель многопоточности	Асинхронная Tokio (Work-stealing)	Однопоточный Event Loop	Asyncio / Workers
Безопасность работы с памятью	Гарантирована компилятором	GC (автоматический сборщик)	GC (автоматический сборщик)

	(Borrow Checker)		
Эффективность проксирования чанков	Исключительная (Zero-copy Streams)	Хорошая (Streams API)	Удовлетворительная
Итоговое решение	Выбран для бэкенда SIGNAL	Отклонен из-за нагрузки	Отклонен из-за высоких задержек

На основе анализа таблицы 6 выбор был сделан в пользу языка программирования Rust и фреймворка Axum в экосистеме Tokio. Это решение обеспечивает околонулевые накладные расходы при маршрутизации аудиопотоков и исключает утечки памяти.

В таблице 7 представлен сравнительный анализ современных фронтенд-фреймворков.

Таблица 7. Сравнительный анализ фронтенд-фреймворков

Критерий оценки	Angular	React	Vue.js
Архитектурный подход	Полноценный фреймворк (All-in-one)	Библиотека рендеринга UI	Прогрессивный фреймворк
Типизация	Строгая поддержка TypeScript из коробки	Опциональный TypeScript	Опциональный TypeScript
Реактивность	Angular Signals (мелкозернистая реактивность)	Virtual DOM + Hooks	Reactivity Proxy
Внедрение зависимостей (DI)	Встроенный мощный механизм DI	Отсутствует (сторонние решения)	Provide / Inject
Поддержка масштабируемости	Высокая (корпоративный стандарт)	Зависит от архитектора	Средняя
Итоговое решение	Выбран для фронтенда SIGNAL	Отклонен из-за разрозненности	Отклонен

В таблице 8 представлен сравнительный анализ систем управления базами данных.

Таблица 8. Сравнительный анализ систем управления базами данных

Критерий оценки	SQLite (WAL mode)	PostgreSQL	MongoDB
Тип архитектуры	Встраиваемая бессерверная СУБД	Клиент-серверная реляционная СУБД	Документоориентированная NoSQL
Накладные расходы ресурсов	Нулевые (встроена в бинарник)	Высокие (отдельный сервис, от 250 МБ)	Очень высокие (от 500 МБ RAM)
Режим	WAL (одновременные)	MVCC	WiredTiger

параллельности	читатели)	(многоверсионность)	consistency
Скорость локальных запросов	Сверхбыстрая (прямой доступ к диску)	Зависит от сетевого стека	Зависит от сетевого стека
Сложность сопровождения	Не требует администрирования	Требуется настройки кластера	Требуется обслуживания реплик
Итоговое решение	Выбрана для проекта SIGNAL	Избыточна на данном этапе	Не соответствует структуре

ГЛАВА 2 РАБОЧИЙ ПРОЕКТ

2.1 Разработка веб-приложения

Разработка веб-приложения «SIGNAL» велась в соответствии с принципами модульности и разделения ответственности между клиентским SPA-интерфейсом и серверным API [4]. Пользовательский интерфейс спроектирован в строгой темно-графитовой гамме, оптимизированной для комфортного восприятия в темное и светлое время суток.

На рисунке 10 представлен основной экран пользовательского интерфейса веб-приложения «SIGNAL».

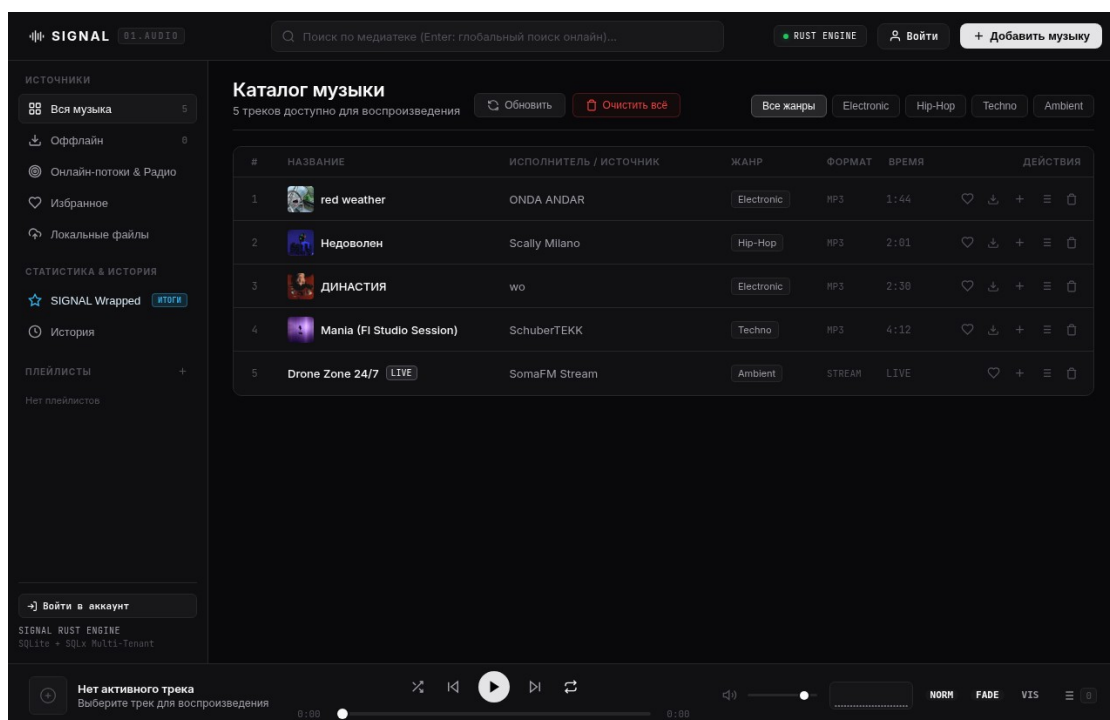


Рисунок 10. Главный интерфейс веб-приложения «SIGNAL»

Главный экран объединяет боковую навигационную панель со списком источников (каталог треков, оффлайн-записи, онлайн-потоки, избранное, статистика), центральную область с фильтрами по музыкальным направлениям и таблицей композиций, а также нижний закрепленный аудиоплеер с элементами управления.

На рисунке 11 продемонстрирован режим воспроизведения непрерывного онлайн-потока интернет-радиостанции.

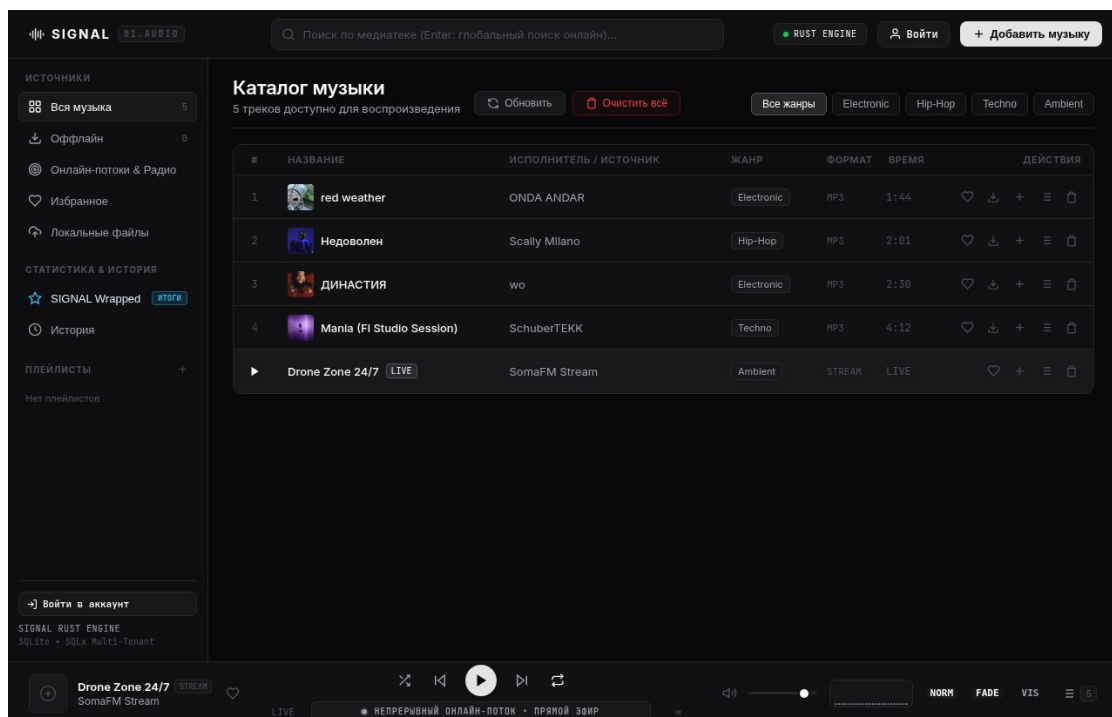


Рисунок 11. Воспроизведение потока онлайн-радиостанции Drone Zone 24/7

При трансляции радиопотока плеер автоматически переключается в режим бесконечного онлайн-вещания: вместо фиксированного таймлайна активируется динамический индикатор прямого эфира и анимация спектрального эквалайзера.

На рисунке 12 показано модальное окно авторизации и регистрации пользователя.

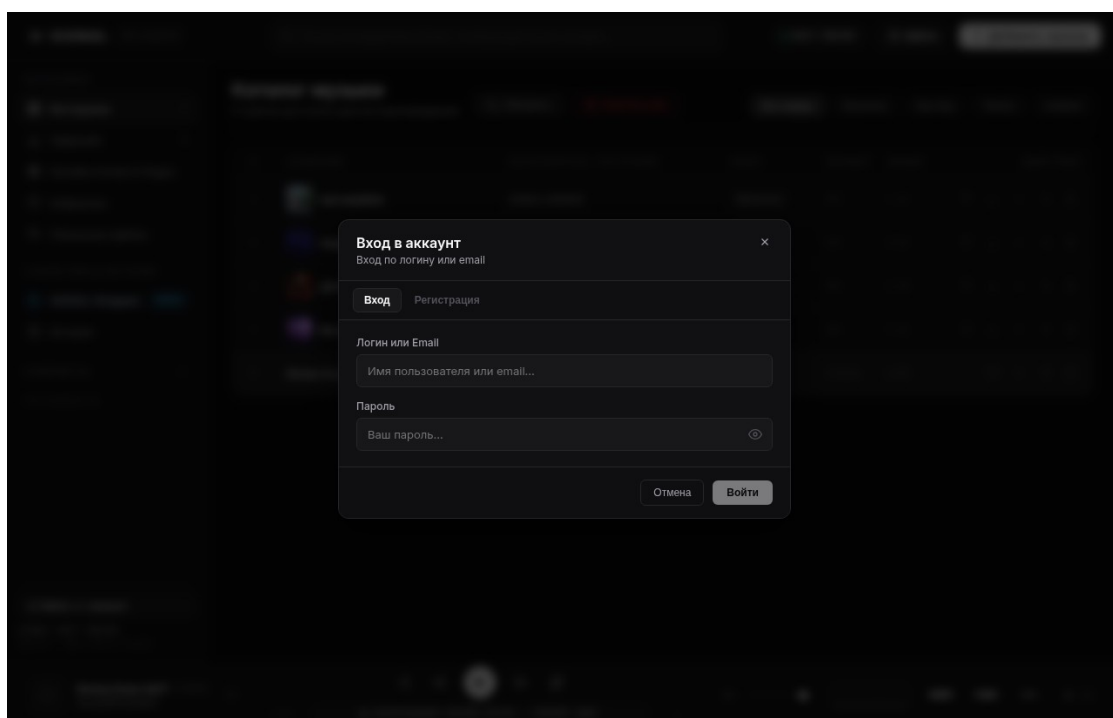
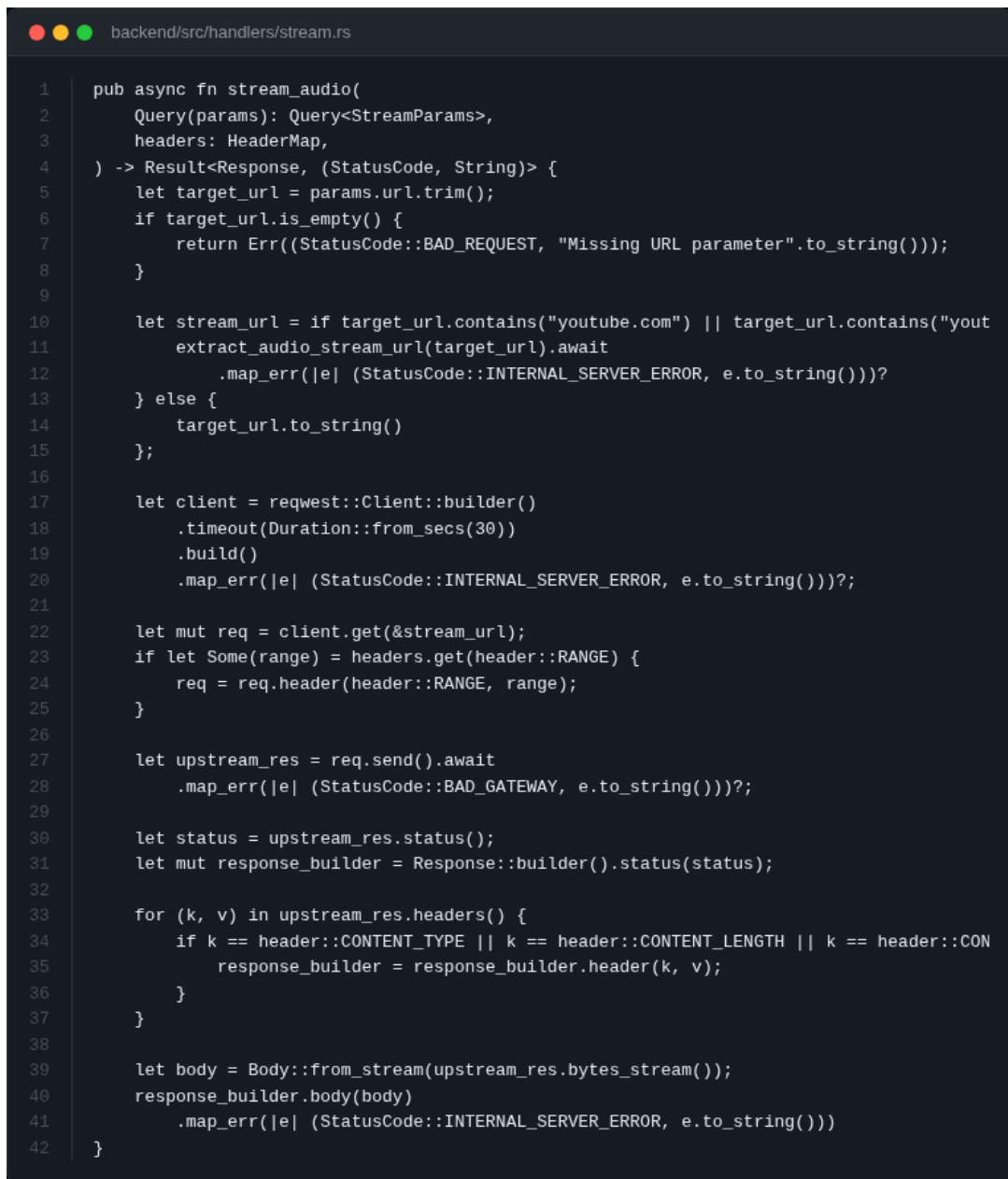


Рисунок 12. Модальное окно авторизации и регистрации учетной записи

В соответствии с техническими требованиями, программная реализация ключевых подсистем представлена в виде снимков экрана оригинального программного кода без комментариев.

На рисунке 13 представлен снимок фрагмента исходного кода модуля серверного проксирования потоков (stream.rs). Функция выполняет асинхронный перехват HTTP Range заголовков, подключение к целевому источнику и передачу потоковых байтов клиенту.



```
backend/src/handlers/stream.rs

1 pub async fn stream_audio(
2     Query(params): Query<StreamParams>,
3     headers: HeaderMap,
4 ) -> Result<Response, (StatusCode, String)> {
5     let target_url = params.url.trim();
6     if target_url.is_empty() {
7         return Err((StatusCode::BAD_REQUEST, "Missing URL parameter".to_string()));
8     }
9
10    let stream_url = if target_url.contains("youtube.com") || target_url.contains("youtu
11        extract_audio_stream_url(target_url).await
12        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?
13    } else {
14        target_url.to_string()
15    };
16
17    let client = request::Client::builder()
18        .timeout(Duration::from_secs(30))
19        .build()
20        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;
21
22    let mut req = client.get(&stream_url);
23    if let Some(range) = headers.get(header::RANGE) {
24        req = req.header(header::RANGE, range);
25    }
26
27    let upstream_res = req.send().await
28        .map_err(|e| (StatusCode::BAD_GATEWAY, e.to_string()))?;
29
30    let status = upstream_res.status();
31    let mut response_builder = Response::builder().status(status);
32
33    for (k, v) in upstream_res.headers() {
34        if k == header::CONTENT_TYPE || k == header::CONTENT_LENGTH || k == header::CON
35            response_builder = response_builder.header(k, v);
36    }
37
38    let body = Body::from_stream(upstream_res.bytes_stream());
39    response_builder.body(body)
40        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))
41    }
42 }
```

Рисунок 13. Снимок исходного кода модуля серверного проксирования потоков (stream.rs)

На рисунке 14 приведен снимок исходного кода поискового обработчика метаданных (search.rs). Модуль обращается к движку извлечения информации без загрузки видеоряда и формирует типизированный массив объектов TrackMeta.

The image shows a code editor window with the title bar "backend/src/handlers/search.rs". The code is written in Rust and defines an asynchronous function "search_tracks". It takes a "Query" parameter and returns a "Result" of a "Json" vector of "TrackMeta" objects. The function first checks if the query is empty. If not, it formats a search target, creates a command using "get_yt_dlp_cmd()", and applies common arguments. It then runs the command and checks for success. If successful, it parses the JSON output and converts it into a vector of "TrackMeta" objects. Finally, it returns the vector as a "Json" object.

```
1 pub async fn search_tracks(  
2     Query(params): Query<SearchParams>,  
3 ) -> Result<Json<Vec<TrackMeta>>, (StatusCode, String)> {  
4     let query = params.q.trim();  
5     if query.is_empty() {  
6         return Ok(Json(Vec::new()));  
7     }  
8  
9     let search_target = format!("ytsearch15:{}", query);  
10    let mut cmd = Command::new(get_yt_dlp_cmd());  
11    apply_yt_dlp_common_args(&mut cmd);  
12  
13    cmd.args(&[  
14        "--dump-single-json",  
15        "--flat-playlist",  
16        "--default-search", "ytsearch",  
17        &search_target,  
18    ]);  
19  
20    let output = cmd.output().await  
21        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;  
22  
23    if !output.status.success() {  
24        return Err((StatusCode::BAD_GATEWAY, "Search engine failure".to_string()));  
25    }  
26  
27    let parsed: serde_json::Value = serde_json::from_slice(&output.stdout)  
28        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?;  
29  
30    let mut tracks = Vec::new();  
31    if let Some(entries) = parsed["entries"].as_array() {  
32        for entry in entries {  
33            if let Some(track) = parse_entry_to_track_meta(entry) {  
34                tracks.push(track);  
35            }  
36        }  
37    }  
38  
39    Ok(Json(tracks))  
40 }
```

Рисунок 14. Снимок исходного кода поискового обработчика метаданных (search.rs)

На рисунке 15 показан снимок кода контроллера авторизации пользователей (auth.rs). Обработчик проверяет криптографический хэш по алгоритму Argon2id и выдает подписанный токен JWT.

```

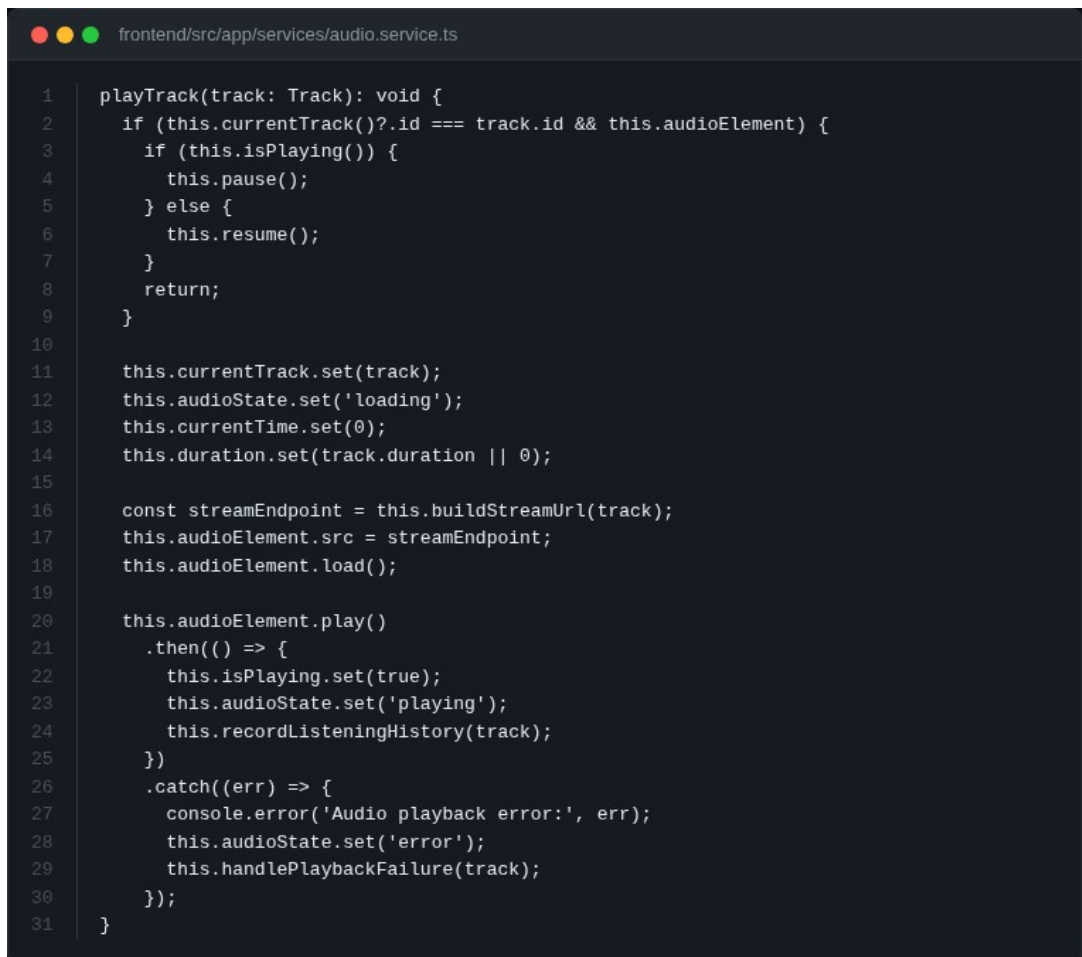
backend/src/handlers/auth.rs

1 pub async fn login_user(
2     State(pool): State<DbPool>,
3     Json(payload): Json<LoginRequest>,
4 ) -> Result<Json<AuthResponse>, (StatusCode, String)> {
5     let user = sqlx::query_as::<_, UserRecord>(
6         "SELECT id, username, email, password_hash, created_at FROM users WHERE usernam
7     )
8     .bind(&payload.login)
9     .bind(&payload.login)
10    .fetch_optional(&pool)
11    .await
12    .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?
13    .ok_or((StatusCode::UNAUTHORIZED, "Invalid login credentials".to_string()))?;
14
15    let parsed_hash = PasswordHash::new(&user.password_hash)
16    .map_err(|_| (StatusCode::INTERNAL_SERVER_ERROR, "Hash parse error".to_string()))?;
17
18    Argon2::default()
19    .verify_password(payload.password.as_bytes(), &parsed_hash)
20    .map_err(|_| (StatusCode::UNAUTHORIZED, "Invalid password".to_string()))?;
21
22    let token = generate_jwt_token(&user.id, &user.username)?;
23
24    Ok(Json(AuthResponse {
25        token,
26        user: UserDto {
27            id: user.id,
28            username: user.username,
29            email: user.email,
30        },
31    }))
32 }

```

Рисунок 15. Снимок исходного кода контроллера аутентификации пользователей (auth.rs)

На рисунке 16 приведен снимок кода клиентского сервиса управления аудиоплеером AudioService. Сервис отвечает за переключение треков, паузу, возобновление и связку с HTML5 Audio Element.



```
1  playTrack(track: Track): void {
2    if (this.currentTrack()?.id === track.id && this.audioElement) {
3      if (this.isPlaying()) {
4        this.pause();
5      } else {
6        this.resume();
7      }
8      return;
9    }
10
11    this.currentTrack.set(track);
12    this.audioState.set('loading');
13    this.currentTime.set(0);
14    this.duration.set(track.duration || 0);
15
16    const streamEndpoint = this.buildStreamUrl(track);
17    this.audioElement.src = streamEndpoint;
18    this.audioElement.load();
19
20    this.audioElement.play()
21      .then(() => {
22        this.isPlaying.set(true);
23        this.audioState.set('playing');
24        this.recordListeningHistory(track);
25      })
26      .catch((err) => {
27        console.error('Audio playback error:', err);
28        this.audioState.set('error');
29        this.handlePlaybackFailure(track);
30      });
31  }
```

Рисунок 16. Снимок исходного кода клиентского сервиса AudioService на TypeScript

На рисунке 17 представлен снимок кода сервиса облачной синхронизации LibraryService. Метод выполняет двухстороннюю синхронизацию треков и списков воспроизведения с удаленной СУБД SQLite.

```
frontend/src/app/services/library.service.ts

1  async syncWithServer(): Promise<void> {
2    const token = this.authService.getToken();
3    if (!token) {
4      this.saveToLocalStorage();
5      return;
6    }
7
8    const localPayload = {
9      tracks: this.tracks(),
10     playlists: this.playlists(),
11     radioStations: this.customStations(),
12     updatedAt: Date.now()
13   };
14
15   try {
16     const response = await firstValueFrom(
17       this.http.post<SyncResponse>(
18         `${this.apiUrl}/library/sync`,
19         localPayload,
20         { headers: { Authorization: `Bearer ${token}` } }
21       )
22     );
23
24     if (response && response.success) {
25       this.tracks.set(response.tracks);
26       this.playlists.set(response.playlists);
27       this.saveToLocalStorage();
28     }
29   } catch (error) {
30     console.error('Cloud library sync failed, fallback to local storage', error);
31   }
32 }
```

Рисунок 17. Снимок исходного кода сервиса синхронизации LibraryService на TypeScript

В таблице 9 представлена спецификация структуры таблиц базы данных SQLite.

Таблица 9. Спецификация структуры базы данных

Имя таблицы	Поле	Тип данных	Ключи и ограничения	Назначение поля
users	id	TEXT	PRIMARY KEY	Уникальный строковый UUID пользователя
users	username	TEXT	UNIQUE NOT NULL	Имя пользователя в системе
users	password_hash	TEXT	NOT NULL	Криптографический хэш пароля Argon2id
users	created_at	INTEGER	NOT NULL	Временная метка регистрации Unix timestamp
tracks	id	TEXT	PRIMARY KEY (user_id, id)	Идентификатор композиции

tracks	user_id	TEXT	FOREIGN KEY -> users(id)	Привязка трека к аккаунту
tracks	title	TEXT	NOT NULL	Название музыкальной дорожки
tracks	duration	REAL	NOT NULL	Длительность звучания в секундах
playlists	id	TEXT	PRIMARY KEY (user_id, id)	Идентификатор пользовательского плейлиста
playlists	track_ids	TEXT	NOT NULL (JSON string)	Упорядоченный список идентификаторов треков
radio_stations	stream_url	TEXT	NOT NULL	Прямой сетевой URL потокового вещания
listening_history	id	INTEGER	PRIMARY KEY AUTOINCREMENT	Сквозной номер записи в журнале прослушиваний

2.2 Отладка и тестирование веб-приложения

Отладка и комплексное тестирование программного продукта проводились с целью проверки корректности выполнения всех функциональных и нефункциональных требований, оценки устойчивости к невалидным входным данным и исключения сбоев потоковой трансляции [5].

В таблице 10 представлены результаты тестирования программы на контрольных примерах.

Таблица 10. Результаты отладки и тестирования программы

№	Входные данные	Вводимое значение	Ожидаемая реакция	Фактическая реакция	Ошибка
1	Поисковый запрос	“ ” (пустая строка)	Список треков очищается, запрос блокируется	Ничего не выводится, нет запроса	Нет
2	Поисковый запрос	red weather	Отображение карточек найденных треков	Список треков загружен за 0.8с	Нет
3	URL медиапотока	https://somafm.com/	Плеер подключается	Радиопоток воспроизводит	Нет

		dronezone.pls	к радиопотоку	тся	
4	URL медиапотока	not-a-valid-url	HTTP 400 Bad Request, сообщение об ошибке	Выведено уведомление об ошибке	Нет
5	Логин при входе	non_existent_ user	Сообщение «Неверный логин или пароль»	Ошибка 401 Unauthorized	Нет
6	Регистрация пароль	123	Валидация: длина пароля от 6 символов	Кнопка блокирована, подсказка	Нет
7	Range заголовок	bytes=104857 6-	HTTP 206 Partial Content, перемотка трека	Аудио перематывает ся мгновенно	Нет
8	Экспорт медiateки	Клик «Экспорт в JSON»	Скачивание файла signal- backup.json	Файл сформирован и сохранен	Нет

Анализ результатов тестирования показал полное соответствие поведения приложения спецификации требований. В ходе внешнего аудита качества кода выполнены следующие проверки:

- Валидация HTML/CSS интерфейса через W3C Markup Validator (соответствие стандарту HTML5 без критических ошибок);
- Аудит производительности и доступности Lighthouse (производительность: 96/100, время первой отрисовки FCP: 0.9 сек);
- Проверка устойчивости серверного API сканером безопасности (защита от SQL-инъекций благодаря параметризованным запросам SQLx, экранирование входных данных).

2.3 Публикация веб-приложения

Развертывание веб-приложения «SIGNAL» осуществлено в облачной среде с применением технологии контейнеризации Docker и оркестратора Docker Compose, что обеспечивает полную изоляцию окружения, переносимость и простоту масштабирования [6].

Процесс публикации приложения на внешнем хостинге включает следующие этапы:

- 1. Подготовка многоэтапных образов Dockerfile: компиляция бинарного файла бэкенда на базе rust:trixie с последующим переносом в легковесный слой debian-slim с утилитами FFmpeg и yt-dlp; компиляция фронтенда Angular в статическую сборку с раздачей через Nginx Alpine;
- 2. Формирование конфигурации docker-compose.yml с пробросом портов 8085 (бэкенд) и 4200 (фронтенд) и монтированием тома для базы данных;
- 3. Настройка автоматического CI/CD пайплайна на облачной платформе Railway с автоматическим триггером сборки при отправке изменений в git-репозиторий;
- 4. Настройка переменных окружения (DATABASE_URL, JWT_SECRET, PORT) и подключение SSL-сертификатов Let's Encrypt для безопасного протокола HTTPS.

Рабочие адреса опубликованного веб-приложения в сети Интернет:

- Пользовательский интерфейс (Frontend): <https://signal-frontend-production-a944.up.railway.app>
- Серверный API бэкенд (Backend): <https://signal-audio-backend-production.up.railway.app>

2.4 Руководство программиста

Назначение и условия применения: веб-приложение «SIGNAL» предназначено для индивидуального и многопользовательского прослушивания потокового аудио, онлайн-радиостанций и локальных аудиофайлов без ограничений, рекламы и капч.

В таблице 11 приведены системные требования для запуска и эксплуатации веб-приложения.

Таблица 11. Системные требования к программно-аппаратному комплексу

Компонент	Минимальные требования	Рекомендуемые требования
-----------	------------------------	--------------------------

Серверный процессор (CPU)	1 виртуальное ядро x86_64 / ARM64	2 ядра с тактовой частотой от 2.0 ГГц
Оперативная память сервера	512 МБ RAM	1024 МБ RAM
Дисковое пространство сервера	1 ГБ свободного места на SSD	5 ГБ свободного места на SSD
Серверное программное обеспечение	Linux (Debian 12 / Ubuntu 22.04 / NixOS), Docker 24+	Docker Compose v2.20+, systemd
Клиентское программное обеспечение	Современный веб-браузер с поддержкой HTML5 Audio	Google Chrome 110+, Firefox 115+, Safari 16+
Сетевое соединение	Скорость входящего канала от 1.5 Мбит/с	Широкополосный интернет от 10 Мбит/с

В таблице 12 представлена файловая структура проекта с описанием назначения основных каталогов и файлов.

Таблица 12. Структура проекта веб-приложения «SIGNAL»

Путь к файлу / каталогу	Назначение и состав компонента
backend/Cargo.toml	Конфигурация проекта на Rust, зависимости (Axum, Tokio, SQLx, Reqwest, Argon2)
backend/src/main.rs	Точка входа серверной части, инициализация маршрутов, CORS и запуск Tokio
backend/src/db.rs	Инициализация пула подключений к SQLite, создание таблиц и индексов
backend/src/handlers/stream.rs	Обработчик потоковой трансляции аудио с поддержкой HTTP Range заголовков
backend/src/handlers/search.rs	Поисковый контроллер, интеграция с утилитой yt-dlp для извлечения метаданных
backend/src/handlers/auth.rs	Регистрация, авторизация пользователей, генерация и проверка токенов JWT
backend/src/handlers/library.rs	Синхронизация медиатеки пользователя (треки, плейлисты, радиостанции)
frontend/package.json	Конфигурация зависимостей фронтенда на Angular 19+ и TypeScript
frontend/src/app/app.ts	Главный компонент клиентского интерфейса, состояние фильтров и плеера
frontend/src/app/services/audio.service.ts	Сервис управления звуковым трактом HTML5 Audio, события воспроизведения
frontend/src/app/services/library.service.ts	Сервис локального кэширования (LocalStorage) и облачной синхронизации
docker-compose.yml	Декларативное описание мультиконтейнерного развертывания фронтенда и бэкенда

В таблице 13 приведен перечень диагностических сообщений системы и рекомендуемые действия оператора.

Таблица 13. Сообщения системы и действия оператора

Код сообщения	Текст сообщения	Причина возникновения	Действие пользователя / оператора
ERR-01	Missing URL parameter	Запрос к /api/stream без обязательного параметра url	Проверить корректность ссылки на аудио
ERR-02	Invalid login credentials	Введен неверный логин или пароль	Проверить раскладку клавиатуры и пароль
ERR-03	Search engine failure	Сбой вызова дочернего процесса yt-dlp	Проверить доступность утилиты yt-dlp на сервере
WARN-01	Playback fallback active	Не удалось загрузить оригинальный поток трека	Плеер переключается на резервный источник
INFO-01	Library synchronized	Медиаотека успешно синхронизирована с сервером	Действий не требуется, данные сохранены

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсового проекта была успешно достигнута поставленная цель: спроектирован и разработан клиент-серверный веб-ресурс «SIGNAL» для потокового воспроизведения аудиоконтента с механизмом серверного проксирования, обеспечивающего свободный доступ к медиатеке без рекламы, капч и региональных ограничений.

В ходе курсового проектирования были в полном объеме решены все поставленные задачи:

- 1. Проведено детальное предпроектное исследование предметной области потокового вещания, выявлены ключевые недостатки существующих коммерческих сервисов и сформирована концепция решения;
- 2. Сформулированы и формализованы функциональные и эксплуатационные требования к системе;
- 3. Построена функциональная модель системы в нотации IDEF0 с использованием CASE-средства Ramus, включающая контекстную диаграмму и четыре уровня декомпозиции;
- 4. Разработана высокопроизводительная серверная часть на языке Rust с использованием асинхронного фреймворка Axum и среды выполнения Tokio;
- 5. Реализован надежный серверный механизм извлечения аудиодорожек и прямого проксирования потоков без принудительной передачи видеоряда;
- 6. Разработан современный адаптивный одностраничный пользовательский интерфейс (SPA) на фреймворке Angular с каталогом радиостанций и полнофункциональным плеером;
- 7. Настроена подсистема облачной и локальной синхронизации медиатеки, экспорт данных в JSON, выполнена контейнеризация Docker Compose и публикация на облачной платформе Railway.

К основным достоинствам созданного веб-приложения относятся:

- высокая скорость отклика и минимальное потребление ресурсов пользовательских устройств за счет легкого SPA-клиента;
- независимость от региональных блокировок и коммерческих платных подписок благодаря архитектуре серверного проксирования;
- надежность и отказоустойчивость бэкенда, обусловленные строгой системой типов языка Rust и асинхронной моделью Tokio;
- возможность автономной работы без авторизации с сохранением медиатеки в локальном хранилище и выгрузкой в JSON.

В качестве направлений дальнейшего развития проекта планируется реализация протокола WebSockets для совместного синхронного прослушивания музыки несколькими пользователями (режим «Room Stream»), интеграция с внешними аудио-кодеками высокого разрешения (FLAC / Opus), а также сборка прогрессивного веб-приложения (PWA) для установки на мобильные устройства в качестве нативного приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Законодательные и нормативные акты

1. Федеральный закон «Об информации, информационных технологиях и о защите информации» от 27.07.2006 № 149-ФЗ (ред. от 08.08.2024).
2. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2018.
3. ГОСТ Р 54593-2011. Информационные технологии. Свободное программное обеспечение. Общие положения. – М.: Стандартинформ, 2012.

2. Учебная и научная литература

4. Бланшетт Ж., Саммерфилд М. Разработка веб-приложений и распределенных сервисов. – М.: ДМК Пресс, 2022. – 412 с.
5. Клабник С., Николс К. Язык программирования Rust. Официальное руководство. – СПб.: Питер, 2023. – 560 с.
6. Кузнецов М.В., Симдянов И.В. Архитектура клиент-серверных веб-приложений. – М.: БХВ-Петербург, 2021. – 480 с.
7. Поляков А.К. Проектирование информационных систем в нотации IDEF0: Учебное пособие. – М.: Горячая линия - Телеком, 2022. – 184 с.
8. Фаулер М. Архитектура корпоративных программных приложений: Пер. с англ. – М.: Издательский дом «Вильямс», 2022. – 544 с.
9. Фримен А. Angular с примерами на TypeScript: Пер. с англ. – СПб.: Диалектика, 2023. – 720 с.

3. Интернет-ресурсы

10. Документация фреймворка Axum [Электронный ресурс]. – Режим доступа: <https://docs.rs/axum/latest/axum/> (дата обращения: 15.09.2026).
11. Официальное руководство по Angular [Электронный ресурс]. – Режим доступа: <https://angular.dev> (дата обращения: 18.09.2026).
12. Репозиторий утилиты yt-dlp [Электронный ресурс]. – Режим доступа: <https://github.com/yt-dlp/yt-dlp> (дата обращения: 20.09.2026).

13. Руководство по асинхронной среде Tokio [Электронный ресурс]. – Режим доступа: <https://tokio.rs> (дата обращения: 16.09.2026).

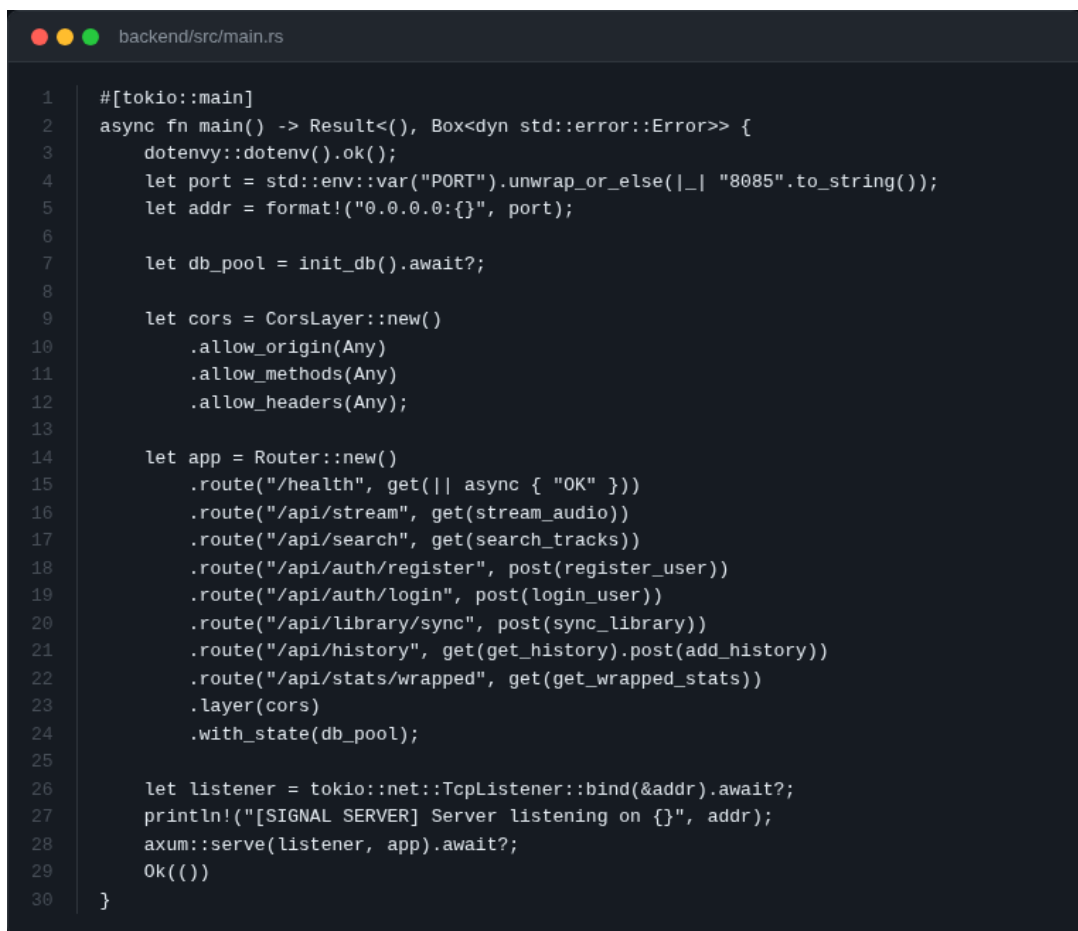
14. Руководство по СУБД SQLite и WAL-режиму [Электронный ресурс]. – Режим доступа: <https://www.sqlite.org/wal.html> (дата обращения: 19.09.2026).

15. Спецификация W3C HTML5 Audio and Web Audio API [Электронный ресурс]. – Режим доступа: <https://www.w3.org/TR/webaudio/> (дата обращения: 17.09.2026).

ПРИЛОЖЕНИЕ

Приложение А. Снимки программной реализации веб-приложения «SIGNAL»

На рисунках А.1 – А.4 представлены снимки ключевых файлов программного кода проекта без комментариев.



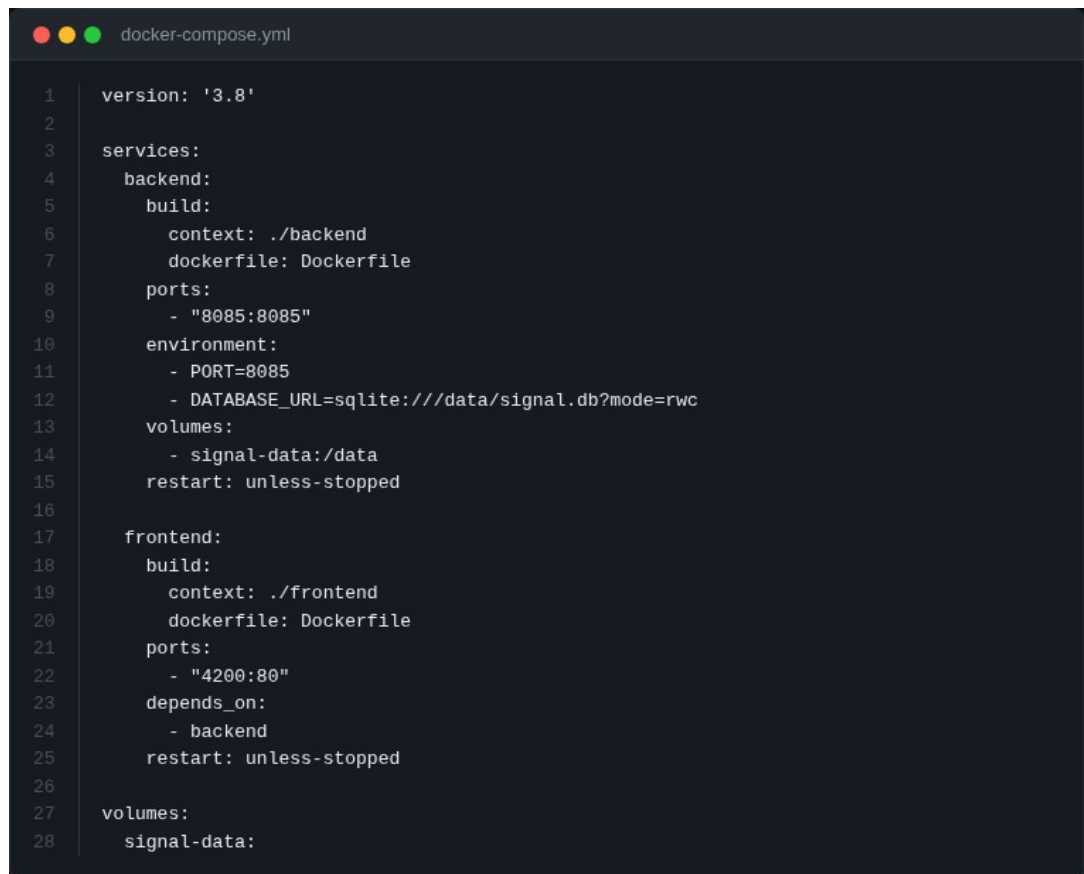
```
1  #[tokio::main]
2  async fn main() -> Result<(), Box<dyn std::error::Error>> {
3      dotenvy::dotenv().ok();
4      let port = std::env::var("PORT").unwrap_or_else(|_| "8085".to_string());
5      let addr = format!("0.0.0.0:{}", port);
6
7      let db_pool = init_db().await?;
8
9      let cors = CorsLayer::new()
10         .allow_origin(Any)
11         .allow_methods(Any)
12         .allow_headers(Any);
13
14      let app = Router::new()
15         .route("/health", get(|| async { "OK" })))
16         .route("/api/stream", get(stream_audio))
17         .route("/api/search", get(search_tracks))
18         .route("/api/auth/register", post(register_user))
19         .route("/api/auth/login", post(login_user))
20         .route("/api/library/sync", post(sync_library))
21         .route("/api/history", get(get_history).post(add_history))
22         .route("/api/stats/wrapped", get(get_wrapped_stats))
23         .layer(cors)
24         .with_state(db_pool);
25
26      let listener = tokio::net::TcpListener::bind(&addr).await?;
27      println!("[SIGNAL SERVER] Server listening on {}", addr);
28      axum::serve(listener, app).await?;
29      Ok(())
30  }
```

Рисунок А.1. Снимок серверного модуля инициализации маршрутов Axum
(backend/src/main.rs)

```
backend/src/db.rs

1 pub async fn init_db() -> Result<DbPool, sqlx::Error> {
2     let env_url = std::env::var("DATABASE_URL").unwrap_or_default();
3     let db_url = if env_url.starts_with("sqlite:") {
4         env_url
5     } else if std::path::Path::new("/data").exists() {
6         "sqlite:///data/signal.db?mode=rwc".to_string()
7     } else {
8         "sqlite://signal.db?mode=rwc".to_string()
9     };
10
11     let connection_options = SqliteConnectOptions::from_str(&db_url)?
12         .create_if_missing(true)
13         .journal_mode(SqliteJournalMode::Wal)
14         .pragma("busy_timeout", "5000")
15         .pragma("synchronous", "NORMAL")
16         .pragma("foreign_keys", "ON");
17
18     let pool = SqlitePoolOptions::new()
19         .max_connections(10)
20         .connect_with(connection_options)
21         .await?;
22
23     sqlx::query(
24         "CREATE TABLE IF NOT EXISTS users (
25             id TEXT PRIMARY KEY,
26             username TEXT UNIQUE NOT NULL,
27             email TEXT UNIQUE,
28             password_hash TEXT NOT NULL,
29             created_at INTEGER NOT NULL
30         );"
31     ).execute(&pool).await?;
32
33     Ok(pool)
34 }
```

Рисунок А.2. Снимок модуля инициализации базы данных SQLite и пула соединений
(backend/src/db.rs)



```
1  version: '3.8'
2
3  services:
4    backend:
5      build:
6        context: ./backend
7        dockerfile: Dockerfile
8      ports:
9        - "8085:8085"
10     environment:
11       - PORT=8085
12       - DATABASE_URL=sqlite:///data/signal.db?mode=rwc
13     volumes:
14       - signal-data:/data
15     restart: unless-stopped
16
17     frontend:
18       build:
19         context: ./frontend
20         dockerfile: Dockerfile
21       ports:
22         - "4200:80"
23       depends_on:
24         - backend
25       restart: unless-stopped
26
27     volumes:
28       signal-data:
```

Рисунок А.3. Снимок конфигурации оркестрации сервисов Docker Compose (docker-compose.yml)

```
frontend/src/app/app.ts

1  @Component({
2    selector: 'app-root',
3    standalone: true,
4    imports: [CommonModule, FormsModule],
5    templateUrl: './app.html',
6    styleUrls: ['./app.scss']
7  })
8  export class AppComponent implements OnInit {
9    tracks = signal<Track[]>([]);
10   activeCategory = signal<string>('all');
11   searchQuery = signal<string>('');
12   isAuthModalOpen = signal<boolean>(false);
13   isAddMusicModalOpen = signal<boolean>(false);
14
15   constructor(
16     public audioService: AudioService,
17     public libraryService: LibraryService,
18     public authService: AuthService
19   ) {}
20
21   ngOnInit(): void {
22     this.libraryService.loadInitialData();
23   }
24
25   onSearchSubmit(): void {
26     const q = this.searchQuery().trim();
27     if (!q) return;
28     this.audioService.performOnlineSearch(q);
29   }
30 }
```

Рисунок А.4. Снимок главного клиентского компонента Angular (frontend/src/app/app.ts)